

Attendance Payroll System User Manual

Last updated: March 24, 2026

1. Purpose

This manual explains how to install, configure, and use the Attendance Payroll System desktop application included in this repository.

The application is a Windows WPF system for:

- administrator sign-in
- employee sign-in
- employee profile management
- attendance time in/time out through a biometric simulation flow
- payroll calculation and payroll record maintenance
- database backup export
- optional school-teacher synchronization
- optional local MySQL offline mirroring

2. Intended Users

This manual is written for three groups:

- **Administrators** who manage employees, attendance, payroll, and backups
- **Employees** who sign in, clock in/out, and review their own records
- **IT/Setup staff** who configure database connections and deployment

3. System Overview

At startup, the app opens the **Login** window.

From there:

- admins open the **Admin Dashboard**
- employees open the **Employee Dashboard**
- the app validates database connectivity before allowing sign-in
- the login screen shows the current database target so users know which server the app is pointing to

The app supports four operating patterns:

1. Direct online MySQL
2. Direct online MySQL with a local MySQL offline mirror
3. Direct online MySQL with school-database teacher sync

4. Legacy Supabase API mode

Direct MySQL is the current default implementation. The legacy Supabase notes are kept in [AttendancePayrollSystem/REMOTE_DB_CONNECTION_GUIDE.md](#).

4. Installation and Launch

4.1 Packaged Windows installer

If your team already built the MSI installer, run it and complete the setup wizard.

The repository's installer pipeline outputs:

- `dist\AttendancePayrollSystem-Setup.msi`

The installer can also launch the app immediately after setup.

4.2 Run from source

If you are running the app directly from the repository, use:

```
dotnet run --project AttendancePayrollSystem\AttendancePayrollSystem.csproj
```

Requirements for source execution:

- Windows
- .NET 8 SDK
- access to the target database server

5. Configuration

5.1 Configuration sources

The app reads settings in this practical order:

1. Existing machine environment variables
2. Laptop-specific database override file saved by the `Database Settings` window
3. Repository `.env`
4. `App.config`

The laptop-specific override file is stored at:

- `%LocalAppData%\AttendancePayrollSystem\database.override.env`

Use the sample file [.env.example](#) as the main reference for `.env` values.

5.2 Required settings for direct MySQL mode

Setting	Required	Purpose
<code>ATTENDANCE_DB_CONNECTION</code>	Yes, unless <code>AttendanceDb</code> is set in <code>App.config</code>	Main online MySQL connection string
<code>ATTENDANCE_BOOTSTRAP_ADMIN_PASSWORD</code>	Yes on first startup, unless <code>BootstrapAdminPassword</code> is set in <code>App.config</code>	Creates the first <code>admin</code> account if none exists

Important notes:

- The initial admin username is always `admin`.
- The bootstrap password should be changed to a strong secret before production use.
- If the app cannot create or reach the main database, sign-in stays disabled.

5.3 Optional settings

Setting	Purpose
<code>ATTENDANCE_OFFLINE_DB_CONNECTION</code>	Local MySQL mirror used when the online DB is unavailable
<code>ATTENDANCE_OFFLINE_AUTO_SYNC</code>	Enables background synchronization of the offline mirror
<code>ATTENDANCE_OFFLINE_SYNC_INTERVAL_SECONDS</code>	Background sync interval, minimum practical value is 10 seconds
<code>SCHOOL_DB_CONNECTION</code>	Enables teacher sync from the school management database
<code>ATTENDANCE_SCHOOL_EMPLOYEES_ONLY</code>	Makes the school DB the exclusive employee source
<code>ATTENDANCE_ENABLE_DEMO_ACCOUNTS</code>	Auto-generates employee login accounts for missing employees
<code>ATTENDANCE_DEMO_EMPLOYEE_PASSWORD</code>	Shared password used for demo employee accounts
<code>SUPABASE_USE_API</code>	Switches the app to legacy Supabase API mode
<code>SUPABASE_URL</code>	Required only in Supabase API mode
<code>SUPABASE_PUBLISHABLE_KEY</code>	Required only in Supabase API mode

Operational notes:

- If `SUPABASE_USE_API=true`, the local MySQL offline mirror is disabled.
- School-teacher sync is also skipped in Supabase API mode.
- `Database Settings` only edits the main online database target. Offline and school-database values still come from `.env`, environment variables, or `App.config`.

6. First Launch

On first startup, the login screen attempts to initialize the database.

If successful:

- the username and password fields are enabled
- the `Current Database Target` card shows the active server and database

If unsuccessful:

- sign-in remains disabled
- a database status message explains the failure
- you can open `Database Settings` to point the laptop to another online MySQL server

6.1 Using Database Settings

From the login screen:

1. Click `Database Settings`.
2. Enter the server/IP, port, database name, username, password, and SSL mode.
3. Click `Test Connection`.
4. Click `Save` if the test succeeds.

Important behavior:

- Saved settings apply only to the current laptop.
- `Use App Default` removes the laptop-specific override and returns the app to the default `.env` or `App.config` target.
- This is useful when different laptops need to point to different database hosts.

7. Accounts and Sign-In

7.1 Admin sign-in

Use:

- Username: `admin`
- Password: the value from `ATTENDANCE_BOOTSTRAP_ADMIN_PASSWORD` or `BootstrapAdminPassword`

If the `admin` account already exists, the app will not recreate it.

7.2 Employee sign-in

Employee access depends on how accounts are provisioned:

- `School sync enabled`: linked teacher accounts are imported from the school system
- `Demo accounts enabled`: missing employee accounts are generated automatically

- `Neither enabled` : employee profiles may exist without matching login accounts

Important limitation:

- The current login screen does not expose self-registration.
- The current desktop UI also does not expose password reset or password management screens.

8. Administrator Workflows

8.1 Admin Dashboard tab

The `Dashboard` tab shows:

- `Total Employees`
- `Present Today`
- `Late Today`
- `Absent Today`
- `Birthday Celebrants Today`
- `Latest Attendance Records`

Buttons:

- `Refresh` reloads dashboard data
- `Backup Database` exports a SQL backup

Notes:

- `Absent Today` is calculated as active employees minus employees with a time-in record today.
- `Birthday Celebrants Today` falls back to hire-date anniversaries if a `BirthDate` column does not exist in the database.

8.2 Employee Management tab

The `Employee Management` tab lets admins:

- view the employee list
- add new employees
- edit selected employees
- delete selected employees
- review the selected employee's details
- review the selected employee's recent attendance records
- open the attendance modal
- open the payroll modal

Add employee

Use the **Add** button and complete:

- employee code
- full name
- email
- phone
- position
- department
- hourly rate
- hire date
- active status

Validation rules:

- employee code and full name are required
- hourly rate must be a valid non-negative number
- hire date is required

Edit employee

Select a row, then click **Edit**.

For school-linked employees, some fields are intentionally locked. In the current UI:

- locked: employee code, full name, email, phone, hire date, active status
- still editable locally: position, department, hourly rate

Delete employee

Select a row, then click **Delete**.

Deletion also removes related:

- attendance records
- payroll records
- linked user account record through the database foreign key

For school-linked employees:

- deleting the local record does not remove the teacher from the school system
- the employee may be recreated on the next successful school sync

Important account note

Creating an employee profile is not the same as creating an employee login.

In the current implementation, employee logins are typically created by:

- school sync
- demo-account generation
- external database provisioning

Do not assume every employee added through **Employee Management** can sign in immediately.

8.3 Attendance modal

From **Employee Management**, select an employee and click **Open Attendance**.

The attendance window contains:

- **Today Summary**
- **Attendance Records** for admins
- **Simulate Fingerprint**

Biometric simulation flow

The **Simulate Fingerprint** button drives the attendance terminal behavior:

- if no record exists for today, it records **Time In**
- if today has a time-in but no time-out, it records **Time Out**
- if both already exist, it reports that attendance is complete

The app records timestamps using the workstation's local date and time. Keep the PC clock accurate.

Attendance status behavior

Important behavior in the current UI:

- a new biometric time-in is saved with status **Present**
- **Late**, **Absent**, and **On Leave** are manual statuses

If your process needs lateness tracking, an admin must update the attendance record status manually.

Attendance record maintenance

Admins can:

- refresh the attendance grid
- select an existing record
- update date, time in, time out, status, and biometric flag
- delete a selected record

Current limitation:

- the desktop UI can edit or delete existing attendance rows
- it does not expose a separate admin form to create arbitrary historical attendance rows from scratch

8.4 Payroll modal

From **Employee Management**, select an employee and click **Open Payroll Modal**.

The payroll window supports:

- payroll calculation by date range
- viewing payroll history
- editing a selected payroll record
- deleting a selected payroll record

Payroll calculation rules

The current calculation logic uses:

- **8.0** regular hours per day
- overtime above 8 hours
- **1.25x** overtime multiplier

The calculator also estimates deductions for:

- SSS
- PhilHealth
- Pag-IBIG
- withholding tax

Statuses available in the UI:

- **Pending**
- **Processed**
- **Paid**
- **Cancelled**

Behavior:

- if a payroll already exists for the selected employee and date range, **Calculate Payroll** updates it
- otherwise, the app creates a new payroll row

8.5 Database backup

Use **Backup Database** from the admin dashboard.

Available backup types:

- **Full Backup**: exports the full schema and all rows
- **Differential Backup**: exports changes since the latest full backup in the same folder
- **Incremental Backup**: exports changes since the latest backup of any type in the same folder

Important rules:

- differential and incremental backups require an earlier backup manifest in the same folder
- keep the generated `.manifest.json` file beside the `.sql` file
- use separate backup folders for different source databases or servers

The current desktop UI exposes backup export only. It does not expose an in-app restore workflow.

9. Employee Workflows

After a successful employee sign-in, the app opens the `Employee Dashboard`.

The employee dashboard shows:

- profile summary
- today's attendance status
- latest payroll summary
- attendance history
- payroll history

9.1 Profile photo

Employees can:

- click `Change Photo`
- select `.png`, `.jpg`, `.jpeg`, or `.bmp`
- remove the photo with `Remove Photo`

Current file limit:

- maximum image size is 5 MB

The employee dashboard currently allows photo management only. It does not expose self-service editing for other profile fields.

9.2 Clock in / clock out

Employees use the `Open Attendance` button from their dashboard.

In employee mode, the attendance window is limited to the terminal flow:

- review today's summary
- simulate fingerprint
- create today's time in
- create today's time out

Employees cannot edit attendance history from their own dashboard.

9.3 Review history

Employees can review:

- recent attendance history
- payroll history
- latest payroll summary card

The dashboard currently loads approximately the last 3 months of attendance history.

10. School Database Sync

If `SCHOOL_DB_CONNECTION` is configured and the app is running in direct MySQL mode:

- teacher records can be synchronized into the employee table
- linked user accounts can also be synchronized

What sync updates for linked teacher records:

- employee code
- full name
- email
- phone
- hire date
- active status
- school linkage identifiers

What remains local for linked teacher records:

- position
- department
- hourly rate

If `ATTENDANCE_SCHOOL_EMPLOYEES_ONLY=true` :

- local employee add/edit/delete actions are locked
- the school database becomes the source of truth for employees

If it is `false` :

- local employees remain allowed
- linked teacher records still refresh from school data

Practical guidance:

- use `Refresh` when you need the latest school-synced employee list
- do not manually rely on local edits to linked identity fields, because school sync can overwrite them

11. Offline Mirror and Synchronization

If `ATTENDANCE_OFFLINE_DB_CONNECTION` is configured, the app can maintain a local MySQL mirror.

Normal online behavior:

- the app uses the online database
- it refreshes the local mirror when possible
- queued offline changes are pushed back to the online database when connectivity returns

Offline fallback behavior:

- if the online database is unavailable
- and the local mirror is reachable
- and the local mirror already contains cached login data
- the app can switch to offline mode

In offline mode:

- users continue working against the local MySQL mirror
- changes are queued for later synchronization

Important limitation:

- first-time offline use is not enough by itself
- the local mirror must already contain cached login data from a prior successful sync

12. Troubleshooting

Problem: sign-in fields are disabled

Likely causes:

- main database connection is not configured
- saved database override points to the wrong server
- online database is down and no valid offline mirror is available

Actions:

1. Open `Database Settings`.
2. Test the online connection.
3. If needed, click `Use App Default`.
4. Confirm `.env` or `App.config` contains a real connection string.

Problem: `admin` cannot log in on a new deployment

Check:

- `ATTENDANCE_BOOTSTRAP_ADMIN_PASSWORD` is set before first startup
- the database is writable
- no earlier `admin` account already exists with a different password

Problem: employee profile exists, but employee cannot log in

Check:

- whether a `UserAccounts` row exists for that employee
- whether school sync is expected to create the account
- whether demo-account generation is enabled
- whether the account or employee profile is inactive

Problem: app reports offline mode

This usually means:

- the online database could not be reached
- the app switched to the local MySQL mirror

Actions:

- restore network/database access
- keep the app running or refresh later so queued operations can synchronize

Problem: differential or incremental backup fails

Check:

- a previous backup exists in the same folder
- the corresponding `.manifest.json` file is still present
- the folder is for the same source server and source database

Problem: late employees are not counted as late

Current behavior:

- the biometric time-in flow saves new attendance rows as `Present`

Action:

- an admin must edit the attendance record and change the status to `Late`

13. Current UI Limits

The current desktop UI does not expose:

- self-registration from the login screen
- password reset or password change screens
- a dedicated in-app restore screen for SQL backups
- a dedicated admin form for creating arbitrary historical attendance rows from scratch

These are operational limits of the current implementation and should be considered during rollout.

14. Repository References

For supporting material in this repository, see:

- [.env.example](#)
- [AttendancePayrollSystem/REMOTE_DB_CONNECTION_GUIDE.md](#)
- [Installer/README.md](#)